

ATIVIDADE PRÁTICA N1 – SISTEMA DE ACERVO DA BIBLIOTECA

Curso: Engenharia de Software **Disciplina:** Programação Orientada a Objetos (ESW448)

Conteúdo: Encapsulamento, Herança, Sobrecarga, Sobreposição, Interfaces e Classes Abstratas

Linguagem: Java

Data: ____/____/____ **Nome:** _____

1. O CONTEXTO

A biblioteca precisa de um programa que controle o acervo. O acervo tem três tipos de item — **livro**, **DVD** e **revista** — e cada tipo tem o seu prazo de empréstimo e o seu valor de multa por dia de atraso. Além disso, livro e DVD podem ser **renovados** pelo aluno; revista, não.

Diferente da atividade anterior, aqui **a modelagem já está pronta e é dada abaixo**. Sua tarefa é implementar exatamente o que está especificado, passo a passo, e fazer o programa rodar.

Esta atividade é 100% prática: tudo o que você precisa decidir são os detalhes de implementação. Evite pular passos — cada um depende do anterior. Se travar, anote a dúvida e traga para a próxima aula.

2. ESPECIFICAÇÃO DAS REGRAS

Tipo	Prazo de empréstimo	Multa por dia de atraso	Renovações permitidas
Livro	14 dias	R\$ 1,00	2
DVD	3 dias	R\$ 2,50	1
Revista	2 dias	R\$ 0,50	não renova

- A multa é calculada como dias de atraso × multa por dia. Sem atraso, a multa é zero.
- Aluno **bolsista** paga metade da multa.
- Todo item nasce **disponível** e passa a **emprestado** quando é emprestado.

3. ROTEIRO — PASSO A PASSO

Passo 1 — A interface `Renovavel`

Crie a interface `Renovavel` com dois métodos (só as assinaturas, sem corpo):

```
boolean renovar();           // tenta renovar; devolve true se conseguiu
int getRenovacoesRestantes();
```

Passo 2 — A classe abstrata `ItemAcervo`

Crie a classe `ItemAcervo` como **abstrata**. Ela é a superclasse dos três tipos de item.

- Atributos, todos **privados**: `codigo` (String), `titulo` (String), `ano` (int) e `emprestado` (boolean).
- Um construtor que recebe código, título e ano. O atributo `emprestado` **não** vem do construtor: começa sempre como `false`.

- Getters para código, título, ano e `isEmprestado()`. **Não crie** `setEmprestado` — a situação do item só muda pelos métodos do Passo 4.

Passo 3 — Os métodos abstratos

Ainda em `ItemAcervo`, declare três métodos **abstratos** (sem corpo). É por eles que cada tipo de item se diferencia:

```
public abstract String getTipo();           // "Livro", "DVD" ou "Revista"  
public abstract int getPrazoDias();  
public abstract double getMultaPorDia();
```

Repare: `ItemAcervo` não tem como responder essas três perguntas — só o tipo concreto sabe. É exatamente por isso que a classe é abstrata e não pode ser instanciada.

Passo 4 — Os métodos concretos (o que é igual para todos)

Ainda em `ItemAcervo`, escreva os métodos **com implementação**, que todas as subclasses vão herdar prontos:

- `calcularMulta(int diasAtraso)` — devolve `diasAtraso * getMultaPorDia()`; se `diasAtraso` for zero ou negativo, devolve 0.0.
- `calcularMulta(int diasAtraso, boolean bolsista)` — **sobrecarga** do método acima: calcula a multa cheia e, se `bolsista` for true, devolve a metade.
- `emprestar()` — marca o item como emprestado; se já estiver emprestado, escreve um aviso na tela.
- `devolver()` — marca o item como disponível.
- `exibirFicha()` — imprime tipo, título, ano, código, prazo, multa por dia e a situação (emprestado ou disponível).

*O `calcularMulta` de dois parâmetros deve **chamar** o de um parâmetro, e não repetir a conta. Regra escrita uma vez só.*

Passo 5 — A subclasse `Livro`

Crie `Livro` estendendo `ItemAcervo` e implementando `Renovavel`:

```
public class Livro extends ItemAcervo implements Renovavel {
```

- Atributos privados: `autor (String)` e `renovacoesRestantes (int, começa em 2)`.
- Construtor recebendo código, título, ano e autor — chamando `super(...)` para os três primeiros.
- Sobreponha (`@Override`) `getTipo()`, `getPrazoDias()` e `getMultaPorDia()` com os valores da tabela da seção 2.
- Sobreponha `exibirFicha()`: chame `super.exibirFicha()` e depois imprima também o autor e as renovações restantes.
- Implemente `renovar()`: se ainda restam renovações, desconta uma e devolve true; senão devolve false.

Passo 6 — As subclasses Dvd e Revista

- Dvd extends ItemAcervo implements Renovavel — atributo próprio duracaoMinutos (int) e 1 renovação. Mesmos métodos do Livro, com os valores do DVD.
- Revista extends ItemAcervo — **sem** implements Renovavel, porque revista não é renovável. Atributo próprio edicao (int). Sobreponha getTipo(), getPrazoDias(), getMultaPorDia() e exibirFicha() (mostrando a edição).

*Este passo é o coração da aula: as três classes herdam da mesma superclasse abstrata, mas **só duas** assinam o contrato Renovavel. Herança e interface respondem a perguntas diferentes.*

Passo 7 — A classe de teste TesteBiblioteca

Crie a classe TesteBiblioteca com o método main fazendo, nesta ordem:

1. Criar um Livro, um Dvd e uma Revista com dados à sua escolha.
2. Emprestar o livro e o DVD (deixe a revista disponível).
3. Guardar os três num **array do tipo da classe abstrata**: ItemAcervo[] acervo = { livro, dvd, revista }; e percorrer com um for, chamando exibirFicha() em cada um.
4. Percorrer o mesmo array de novo, imprimindo, para 4 dias de atraso, a multa comum e a multa de bolsista de cada item (usando as duas versões de calcularMulta).
5. Guardar apenas o livro e o DVD num **array do tipo da interface**: Renovavel[] renovaveis = { livro, dvd }; e, para cada um, chamar renovar() **duas vezes**, imprimindo o resultado e as renovações restantes.

4. CONFERINDO O RESULTADO

Se a sua implementação estiver correta, com 4 dias de atraso os valores impressos no item 4 do Passo 7 devem ser:

Item	Multa comum	Multa bolsista
Livro	R\$ 4,00	R\$ 2,00
DVD	R\$ 10,00	R\$ 5,00
Revista	R\$ 2,00	R\$ 1,00

E no item 5, o livro deve renovar duas vezes com sucesso (true, true) e o DVD deve renovar uma vez (true) e falhar na segunda (false).

5. TESTE DE COMPREENSÃO (faça no código)

Depois que o programa estiver rodando, faça os dois testes abaixo e **deixe o resultado comentado no código**:

- Escreva ItemAcervo item = new ItemAcervo("X-1", "Teste", 2026); e tente compilar. Anote a mensagem de erro e explique em uma linha por que ela acontece.
- Coloque a Revista dentro do array Renovavel[] e tente compilar. Anote o erro e explique por quê.

Depois **comente ou apague** essas duas linhas para que o programa volte a compilar.

6. O QUE SERÁ AVALIADO

Critério	O que se espera
Encapsulamento	atributos privados, acesso por getters, sem <code>setEmprestado</code>
Classe abstrata	<code>ItemAcervo</code> abstrata, com métodos abstratos e concretos convivendo
Herança	três subclasses reaproveitando o código de <code>ItemAcervo</code> , com <code>super (...)</code>
Sobreposição	<code>@Override</code> dos métodos abstratos e de <code>exibirFicha()</code> usando <code>super.exibirFicha()</code>
Sobrecarga	as duas versões de <code>calcularMulta</code> , com a segunda chamando a primeira
Interface	<code>Renovavel</code> implementada só por <code>Livro</code> e <code>Dvd</code>
Polimorfismo	os dois arrays do Passo 7 (<code>ItemAcervo[]</code> e <code>Renovavel[]</code>) percorridos com <code>for</code>
Execução	programa compila, roda e bate com a tabela da seção 4

7. DESAFIO OPCIONAL (para quem terminar antes)

Acrescente ao acervo um quarto tipo, `Ebook`, que **não** ocupa prateleira: prazo de 7 dias e multa zero (não existe atraso, o acesso simplesmente expira). Ele é renovável, com renovações ilimitadas. Pense: o que muda no `calcularMulta` herdado? Você precisou sobrepor alguma coisa a mais?

8. ENTREGA

Envie os arquivos `.java` (interface, classe abstrata, três subclasses e classe de teste), prontos para compilar e executar, junto com a saída do programa. Trabalho individual.

Prazo de entrega: 29/08/2026 (sábado).